

Self-Play Reinforcement Learning for Polytopia

Alan K. Mayer, Christian O’Connor, Markus Gruendler,
Matthew Lu, Kinish Sathish, and Tanmay Deshmukh

Abstract. Turn-based strategy games are challenging environments for reinforcement learning because they combine long-horizon planning, large action spaces, resource management, and partially observable maps. In this project, we study reinforcement learning agents for *Polytopia* through the open-source *Tribes* research environment. We implement and evaluate several agents, including Deep Q-Networks, Proximal Policy Optimization, Advantage Actor-Critic, and an AlphaZero-inspired self-play agent. Our agents are trained to optimize score-based gameplay and are evaluated against random agents, heuristic search agents, and Monte Carlo Tree Search baselines. We find that several neural agents can reliably outperform random play, while stronger search-based baselines remain difficult to beat. Our results highlight both the promise and limitations of applying modern self-play methods to large-scale strategy games.

1 Introduction

Self-play reinforcement learning has produced major advances in complex games. Systems such as AlphaGo Zero and AlphaZero showed that agents can learn strong policies from repeated self-play by combining neural networks with search-based policy improvement [2]. Subsequent deep reinforcement learning methods motivated the application of search-augmented decision making, policy gradient learning, and value function approximation to more open-ended strategy environments.

1.1 Polytopia

Polytopia is a turn-based strategy game played on a procedurally generated map. Players build cities, harvest resources, develop technologies, explore territory, and manage military conflicts with rival civilizations. In the score game mode, the winning civilization naturally has the highest final score, which can be increased through territory capture, city growth, infrastructure construction, and technological progress. These mechanics make Polytopia a difficult reinforcement learning environment: the state space is large, the action space varies at every turn, and victory often requires long-term strategic planning.

1.2 Related Work

We use *Tribes*, an open-source re-implementation of Polytopia designed for AI research [1]. *Tribes* provides a useful benchmark setting because it also includes

several hand-crafted heuristic and search-based agents, including One-Step Look-Ahead, Monte Carlo, Monte Carlo Tree Search, and Rolling-Horizon Evolutionary Algorithm methods. These agents are much stronger than baselines like random play, but rely heavily on manually designed search and evaluation strategies.

Our AlphaZero-inspired approach combines neural policy/value networks with Monte Carlo Tree Search (MCTS) to improve policies through self-play [2]. In AlphaZero, the neural network predicts both a policy distribution over actions and a value estimate for the current state. MCTS then uses these predictions to produce improved action targets and the model is trained to imitate the improved policy while predicting game outcomes.

1.3 Proposed Agents

In this project, we investigate whether deep reinforcement learning agents can perform well in this environment. We implement four agents: Deep Q-Networks (DQN), Proximal Policy Optimization (PPO), Advantage Actor-Critic (A2C), and an AlphaZero-inspired self-play agent. The first three serve as neural reinforcement learning baselines, while the AlphaZero-inspired model explores whether self-play and policy/value learning can scale to Polytopia-like gameplay. Our results show that learned agents outperform random baselines, while search-based agents remain challenging opponents. We also identify practical difficulties caused by the extremely large branching factor, partial observability, and map imbalance.

2 Materials and Methods

2.1 Environment and Objective

Our task is to train agents for the score game mode in Polytopia through the Tribes environment. Each agent controls a civilization and attempts to maximize final score by end of game. The agent must choose actions from training and maneuvering troops, harvesting resources, developing infrastructure, researching technologies from the technology tree, and expanding their territory.

The environment is difficult because the legal action set changes from turn to turn. Unlike fixed-action games, a Polytopia agent cannot simply output one action from a small static set. Instead, the agent must reason over a large and variable set of possible actions. This issue is especially important for policy-gradient and actor-critic methods, where invalid actions must be masked or encoded separately.

2.2 Deep Q-Network

Our DQN agent uses a simple multilayer perceptron to map an encoded global game state to Q-values over possible actions. The network contains two fully connected hidden layers of size 256 with ReLU activations. During training, the

agent learns to estimate the expected return of each action and chooses actions using an exploration strategy.

DQN provides a simple value-based baseline, but it is not naturally suited to Polytopia’s large and changing action space. Since the set of legal actions varies by state, representing all possible actions as a fixed Q-value vector requires either heavy action masking or a simplified action representation. As a result, DQN is useful as a baseline but is less flexible than the actor-critic methods.

2.3 Proximal Policy Optimization

The PPO agent employs a dual-encoder actor-critic architecture designed for variable action spaces. The game state is encoded into a fixed-length feature vector and processed by a state network consisting of two fully connected layers with Layer Normalization and GELU activations, optionally followed by residual blocks. Legal actions are encoded separately with a feature representation that combines a one-hot encoding of the action type with hashed textual features extracted from the action description. The resulting joint representation is passed through a policy head that produces a score for each legal action. A masked categorical distribution is then constructed over the available actions, ensuring that only legal actions can be selected. The critic shares the state encoder and estimates the state value through a separate linear value head.

Training follows the standard Proximal Policy Optimization (PPO) framework with clipped policy updates, value-function regression, and entropy regularization. Advantages are computed using Generalized Advantage Estimation (GAE) with discount factor $\gamma = 0.99$ and $\lambda = 0.95$. Gradients are clipped to stabilize optimization, and action selection during training is performed by sampling from the learned policy distribution. In our experiments, the PPO agent was able to outperform random agents, although it remained weaker than some of the stronger search-based baselines.

2.4 Advantage Actor-Critic

Our A2C agent uses separate actor and critic components. The actor embeds both the game state and text-based legal action features, then combines these embeddings to score legal actions. The critic is a separate multilayer perceptron with two hidden layers, LayerNorm, and GELU activations, and it predicts the expected return of the current state.

This architecture is better matched to the variable action space because the actor scores the available legal actions directly rather than assuming a fixed action set. In our experiments, A2C was one of the strongest learned baselines. It performed extremely well against random agents and was competitive with some search-based methods, although it still struggled against stronger MCTS variants.

2.5 AlphaZero-Inspired Self-Play Agent

Our AlphaZero-inspired agent uses a Transformer-based architecture to encode the game state. The model represents map features, units, cities, terrain, economy, and technology progression as embeddings that can interact through attention. The network outputs both a policy distribution over actions and a value estimate for the current state.

The intended training loop follows the AlphaZero framework: the agent generates games through self-play, stores resulting trajectories in a replay buffer, and trains the policy and value heads from self-generated data. In principle, MCTS can be used as a policy improvement operator by using the current network to guide search, then training the policy head to imitate the improved search distribution.

2.6 Methodological Challenges

In practice, deep learning approaches were difficult to apply effectively to Polytopia. The branching factor can be extremely large because a single turn can contain many possible combinations of unit interactions and territorial development, further complicated by features unlocked in the technology tree.

It is worth noting that procedurally generated maps can end up being highly imbalanced: a civilization can start off stranded in the ocean with minimal access to resources while another ends up with free reign of the rest of the map. When one player is favored from the initial map layout, the value network can quickly learn map bias rather than whether actions contributed meaningfully, and it is in these cases that MCTS may improve the policy only slightly because many actions appear similarly strong given an initial map advantage.

Additionally, we did not fully train and evaluate models under partial observability as is the case in Polytopia. This is an important distinction, because while Polytopia is only partially observable, our primary benchmarks from the Tribes paper operate under similar simplifying assumptions. The additional complexity of fog-of-war learning is left as future work instead of being the focus of this project.

3 Results

3.1 Performance Against Random Agents

All learned agents were first evaluated against random agents. This baseline tests whether the agent learns basic positive behaviors such as expanding cities, training units, and avoiding obviously bad moves.

DQN achieves an overwhelming win rate against random play and a significantly higher average final score than the random baseline. This suggests that even a relatively simple neural network can learn a reasonably correct state valuation for this environment.

PPO similarly outperformed random play and demonstrates that its policy-gradient and actor-critic structure were capable of learning a reasonably strong policy. This result holds for implementations with and without residual blocks. Because the residual block architecture didn’t demonstrate meaningful improvements over simpler and shallower networks (which performed better when adjusting for similar compute), the residual PPO results are omitted.

A2C is no exception and convincingly wins nearly all games played against random agents, which in turn reflects the strength of its legal-action scoring architecture.

3.2 Performance Against Search-Based Agents

Our neural agents were further evaluated against the following search-based agents over the course of 50 games: pure MCTS and heuristic MCTS each with rollouts, as well as heuristic MCTS without rollouts. These baselines are substantially stronger than random play because they simulate how actions or specific metrics influence future outcomes, and backpropagate their signal (either score or the custom heuristic value) along the actions taken.

Against the heuristic MCTS without rollouts, learned agents were sometimes competitive, but the results varied by model. A2C and PPO showed meaningful performance, while AlphaZero-inspired play produced mixed results depending on the map and opponent.

Against the heuristic MCTS with rollouts, the learned agents seemed to struggle. This is expected because rollouts provide MCTS with additional information about the long-term consequences of actions before it uses its heuristic value function.

The heuristic MCTS agents found the A2C agent to be their strongest competitive match, and the non-rollout agent even lost more games than it won to A2C. This suggests A2C was able to score actions well enough to compete with the hand-crafted heuristic function used by the MCTS agents.

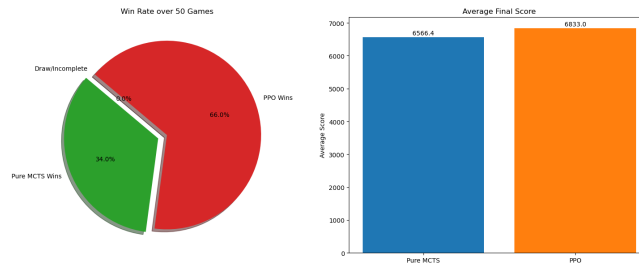


Fig. 1: 2 Layer non-Residual PPO vs Pure MCTS with rollouts, 50 games

The pure MCTS agent found its strongest opponent in the PPO agent, losing substantially more games than it won. This suggests that the PPO agent’s

learned policy captured useful strategic behavior better than the pure MCTS agent could by simulating additional steps.

3.3 Comparison Between Learned Agents

The learned agents also differed substantially from one another. DQN provided a useful baseline and could beat random play, but it was limited by its fixed action representation. PPO was more stable and used an actor-critic structure, but its results depended heavily on training quality. A2C performed best against random play and remained competitive in some search-agent comparisons, likely because its architecture directly scored legal actions. The AlphaZero-inspired agent was conceptually the closest to our self-play goal, but its performance was limited by the difficulty of using MCTS effectively in such a large action space.

3.4 AlphaZero-Inspired Agent Analysis

The AlphaZero-inspired agent performed well in some learned-agent comparisons, such as against DQN, but the full AlphaZero-style training loop was difficult to scale. The main difficulty was that MCTS did not always provide strong policy improvement. In classic board games, MCTS can search a manageable action space and often identifies clearly stronger moves. In Polytopia, however, the branching factor is much larger, and map imbalance can dominate the value signal.

A key failure mode was that the value function could learn that one player was favored due to map layout, almost regardless of the specific moves chosen. In that setting, search may distinguish obviously harmful moves from reasonable moves, but it may not provide a fine-grained improvement signal among many plausible strategic actions. As a result, the policy may learn mostly non-detrimental near-random behavior rather than strong long-term strategy.

3.5 Discussion

The results suggest that neural reinforcement learning can produce useful Polytopia agents, especially against weak baselines. However, beating random agents is not enough to demonstrate strong strategic competence. Search-based baselines remain important because they test whether learned policies can compete with planning methods.

Our experiments also show that Polytopia has several properties that make standard self-play difficult. First, the action space is large and state-dependent. Second, the game has long-term credit assignment: early choices about exploration or technology can affect outcomes many turns later. Third, procedural map generation introduces imbalance, which can make value learning noisy. Finally, partial observability proved too complex to be worth the investment, while limiting the relevance of a comparison to the search-based agents.

4 Conclusion and Future Work

This project explored reinforcement learning and self-play methods for Polytopia using the Tribes environment. We implemented DQN, PPO, A2C, and AlphaZero-inspired agents and compared them against random and search-based baselines. Our learned agents were able to outperform random play, and A2C and PPO in particular showed strong performance against particular MCTS agents. In general, however, the stronger MCTS-based agents remained difficult to beat consistently.

Future work could improve performance in several directions. Training under full fog of war would make the environment more closely match the game and provide insight into how the different agents learn under alternative decision-making conditions. Supervised pretraining from high-quality gameplay data could give the policy and value networks a stronger starting point before self-play. The value target could be modified to incorporate the final score margin rather than only win/loss, giving the model a more granular learning signal, and more compute would certainly be helpful since larger models and longer training runs can help agents learn more sophisticated strategic patterns.

Another promising direction is improving the integration of MCTS. Instead of applying search directly over the full enormous action space, future systems could use hierarchical action selection, action pruning, or learned candidate-action generation, all of which are methods that could enhance MCTS as a policy improvement operator.

Overall, our work is an initial step toward applying modern self-play reinforcement learning to complex turn-based strategy games. While Polytopia remains challenging, the results show that learned agents can acquire meaningful gameplay behavior and provide a foundation for future work on scalable strategy-game reinforcement learning.

Bibliography

- [1] Perez-Liebana, D., et al.: Tribes: A New Turn-Based Strategy Game for AI Research. In: Proceedings of the IEEE Conference on Games (2020)
- [2] Silver, D., Hubert, T., Schrittwieser, J., et al.: A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science* 362(6419), 1140–1144 (2018)
- [3] Becker, F., et al.: Simultaneous AlphaZero: Extending Tree Search to Markov Games. Preprint (2025)